

Part 1
Taught you
the Basics...
Part 2 takes you
to the next level!

JAVA

PART 2

Same
Language
More Power

OOP + CORE CONCEPTS

PART 2

NOW **JAVA** GETS INTERESTING 🙄



Think
Build
Solve
Grow
...



Better
Concepts
Stronger
Developers
...

Classes → Objects → Constructors

Inheritance → Polymorphism

Abstraction → Encapsulation

IN THIS PART ↓

- ✓ Classes & Objects
- ✓ Constructors + this
- ✓ Inheritance
- ✓ Polymorphism
- ✓ Abstraction
- ✓ Interfaces
- ✓ Encapsulation
- ✓ Access Modifiers
- ✓ Exception Handling
- ✓ Collections
- ✓ Multithreading
- ✓ File Handling
- ✓ Interview Questions



Can you answer these?

- Why can't constructors be inherited?
- What actually happens during method overriding?
- Why does Java need interfaces?
- Overloading vs overriding – what's REALLY different?
- Why is encapsulation important?

Good
Developers
Understand
Why
Not Just
What



INTERVIEW MODE

These aren't just Java concepts.
These are the questions interviewers
keep testing.

Missed Part 1?

Start with Java Fundamentals
before this part.

PART 1 → Pages 1-8



Same
Language
Bigger
Opportunities
...

SWIPE →

Your OOP journey starts here.



SAVE THIS

Use these pages for
Java interview revision.

Code Today
Better Tomorrow...
...

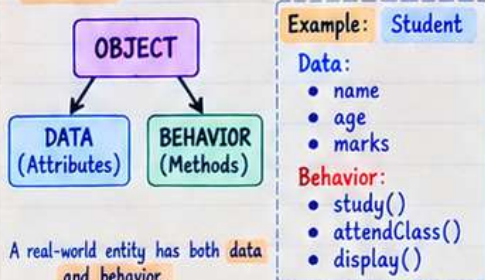


9. OOP CONCEPTS IN JAVA

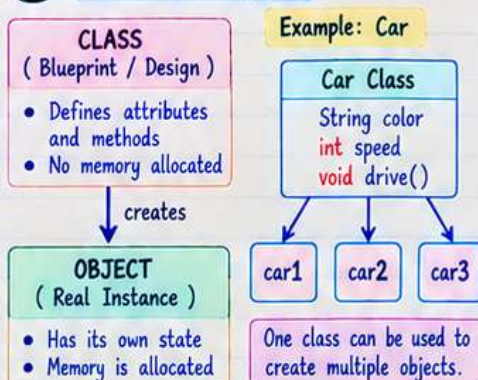
Model real-world entities using objects, data and behavior.

① What is OOP ?

OOP stands for Object-Oriented Programming. It is a programming approach where programs are designed around objects that contain **data** and **behavior**.



② Class vs Object



③ Four Core OOP Principles

- Encapsulation** → Bundle data and control access.
- Inheritance** → Reuse from existing class.
- Polymorphism** → Same interface, different behavior.
- Abstraction** → Show essential, hide details.

"These four principles make Java powerful, flexible and easy to maintain."

④ Encapsulation in Code

```
class BankAccount {
    private double balance;
    void deposit(double amount) {
        if (amount > 0)
            balance += amount;
    }
    double getBalance() {
        return balance;
    }
}
```

private balance → direct external access restricted

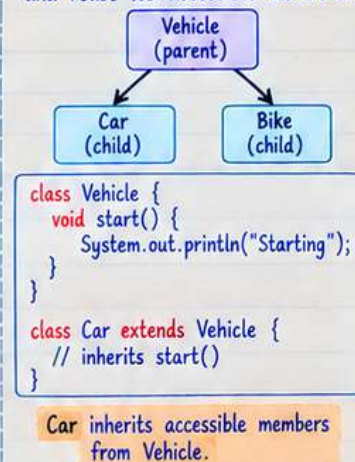
deposit() → controlled modification

getBalance() → controlled reading

Encapsulation = data + controlled access

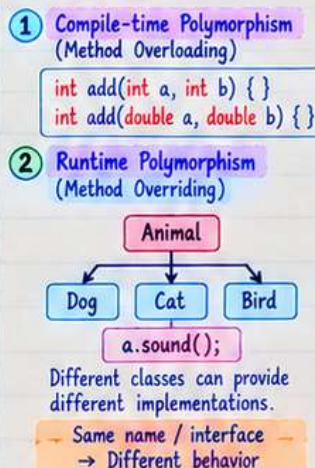
⑤ Inheritance

Inheritance allows a new class to be created from an existing class and reuse its accessible members.



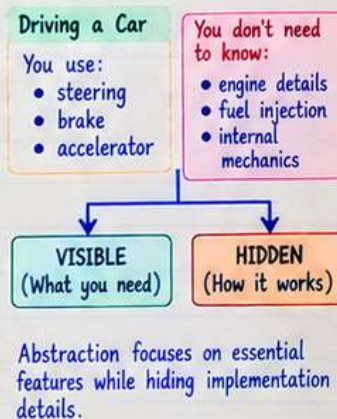
⑥ Polymorphism

One method name can have different behaviors depending on the situation.



⑦ Abstraction

Abstraction shows only the essential features and hides unnecessary implementation details.



⑧ Association

Association represents a relationship between two independent objects.



⑨ OOP Quick Comparison

CONCEPT	MAIN IDEA	SIMPLE KEYWORD
Encapsulation	Control access to data	private
Inheritance	Reuse through hierarchy	extends
Polymorphism	Different behavior	overload / override
Abstraction	Hide implementation details	abstract / interface

⑩ Easy Way to Remember

Encapsulation	ATM hides account internals
Inheritance	Car is a Vehicle
Polymorphism	Same action, different behavior
Abstraction	Use a device without knowing its internal mechanism

⑪ Interview Surprise

- Q:** What are the four pillars of OOP?
- A:** Encapsulation, Inheritance, Polymorphism, Abstraction.
- Q:** Which OOP concept is mainly related to data hiding?
- A:** Encapsulation.

- Q:** Which concept allows one method name to behave differently?
- A:** Polymorphism.

⇒ These concepts are frequently asked in Java interviews!

⑫ Remember

- Encapsulation → Protect/control data
- Inheritance → Reuse relationship
- Polymorphism → Many forms
- Abstraction → Hide implementation details
- Class → Blueprint
- Object → Instance

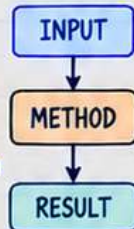
METHODS + PARAMETERS

Break complex programs into small, reusable tasks.

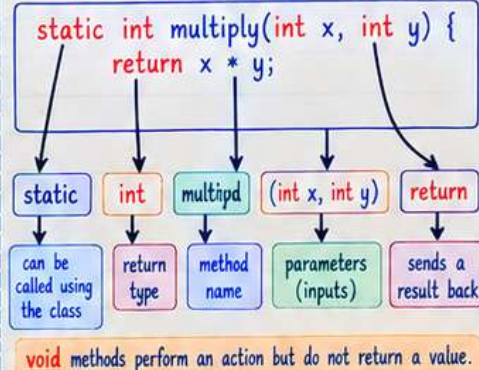
① What is a Method?

A method is a named block of code that performs a specific operation.

- Improves code reuse
- Makes programs easier to read
- Helps divide a large problem into smaller tasks



② Method Syntax



③ Types of Methods

Static Method

- Called using the class.
- Example: `Math.max(4, 9);`

Instance Method

- Called using an object.
- Example: `obj.show();`

Parameterized Method

- Receives input values.
- Example: `sum(5, 7);`

Returning Method

- Sends a result back to the caller.
- Example: `int result = calculate();`

④ Method Overloading

Method overloading means using the same method name with different parameter lists.

```

void print(int value) { }
void print(String value) { }
void print(int a, int b) { }
  
```

Same Name + Different Parameters = **OVERLOADING**

! IMPORTANT:

Changing only the return type does NOT create a valid overloaded method.

```

int print(int a) { }
double print(int a) { } ❌ Not allowed
  
```

⑤ Parameters vs Arguments

PARAMETER	ARGUMENT
- The variable written in the method definition. - Example: <code>int add(int a, int b) { return a + b; }</code>	- The actual value supplied when calling the method. - Example: <code>add(10, 20);</code>
<code>a, b</code> → Parameters	<code>10, 20</code> → Arguments

Parameters are defined in the method.
Arguments are passed during the method call.

⑥ Pass-by-Value

Java passes a copy of the value to a method.

```

static void change(int x) {
    x = 99;
}

int a = 10;
change(a);
System.out.println(a); // 10
  
```

After method call:
`a = 10`
(unchanged)

JAVA IS ALWAYS PASS-BY-VALUE.

- For primitive types, a copy of the value is passed.
- For objects, a copy of the reference is passed (the reference value is copied).

⑦ Recursion

Recursion occurs when a method calls itself to solve a smaller version of the same problem.

```

static int sum(int n) {
    if (n == 1) return 1; // base case
    return n + sum(n - 1); // recursive case
}
  
```

How it works for `sum(4)`:

```

sum(4)
  ↓
4 + sum(3)
  ↓
4 + 3 + sum(2)
  ↓
4 + 3 + 2 + sum(1)
  ↓
10
  
```

BASE CASE

- Stops the recursion.
- In this example: `n == 1`

RECURSIVE CASE

- Calls the method again with a smaller value.
- In this example: `sum(n - 1)`

★ INTERVIEW TIP

Q: Can Java overload methods by changing only return type?

A: No. The parameter list must be different.

Method overloading is based on the method name and parameter types, not on the return type.

📌 REMEMBER

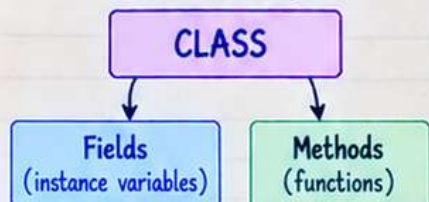
- Method → reusable task
- Parameter → definition input
- Argument → calling value
- Overloading → same name + different parameters
- Recursion → base case + recursive case

CLASSES + OBJECTS

Class = Blueprint | Object = Real Instance

① What is a Class ?

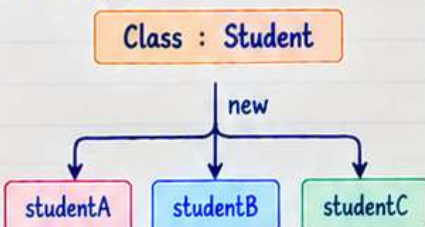
A class is a blueprint that defines the data and behavior an object can have.



- Fields describe the state (data).
- Methods describe the behavior (operations).

② What is an Object ?

An object is a runtime instance of a class.



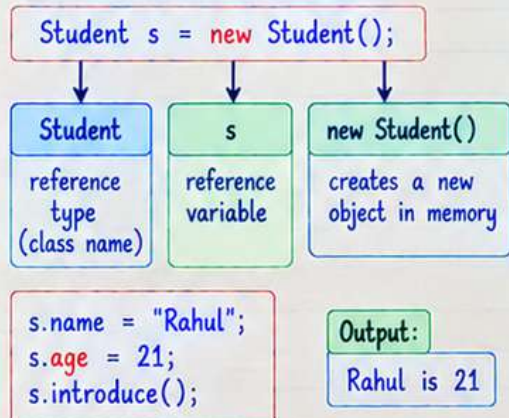
Different objects can have different field values while sharing the same class structure.

③ Class Syntax

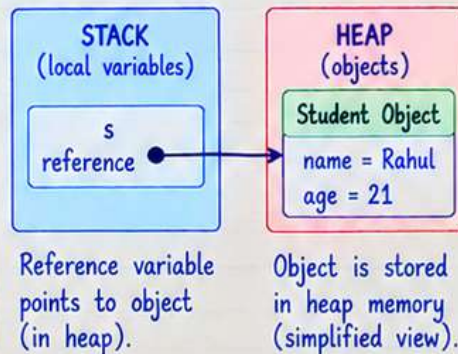
```
class Student { ← class name
    String name; } → fields
    int age;      (instance variables)

    void introduce() { ← method
        System.out.println(name + " is " + age);
    }
}
```

④ Creating an Object



⑤ Memory Idea



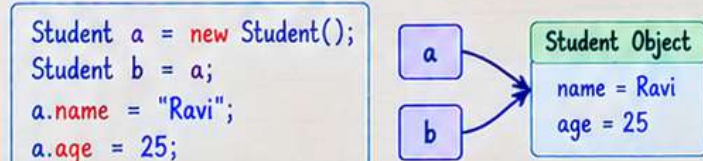
⑥ Multiple Objects

```
Student s1 = new Student();
Student s2 = new Student();
s1.name = "Asha";
s2.name = "Vikram";
s1.age = 20;
s2.age = 22;
```



Same class, different object state.

⑦ Reference Assignment



Both reference variables now refer to the same object.

`a == b` → true (Both point to same object)

⑧ Class vs Object

	CLASS	OBJECT
Meaning	Blueprint for objects	Actual instance of a class
Exists	At compile time (logical definition)	At runtime
Contains	Fields and methods	Has its own state (field values)
Purpose	Used to create objects	Represents a real entity in memory
Example	<code>class Student { ... }</code>	<code>Student s = new Student();</code>



INTERVIEW SURPRISE

Q: What does 'new' do?

A: 'new' requests creation of n object and returns a reference to that object.



REMEMBER

- Class → Blueprint
- Object → Instance
- Field → State (data)
- Method → Behavior (function)
- Reference → Points to an object

12. CONSTRUCTORS + this KEYWORD

Constructors initialize objects when they are created. The **this** keyword refers to the current object.

① What is a Constructor ?

- A constructor is a special member used to initialize an object.
- It has the same name as its class.
- It does not have a return type.
- It runs when an object is created.
- Constructors can be overloaded.

SAME NAME AS CLASS

NO RETURN TYPE

② Basic Constructor Syntax

```
class Employee {
    Employee() {
        System.out.println("Object created");
    }
}
```

new Employee()

constructor runs

object initialized

Employee e = new Employee();

③ Types of Constructors

TYPE	PURPOSE	EXAMPLE
No-arg constructor	Takes no arguments	Employee()
Parameterized constructor	Receives values	Employee(String name)
Constructor overloading	Multiple constructors with different parameter lists	Employee() Employee(String name)

④ Parameterized Constructor

```
class Employee {
    String name;
    int id;

    Employee(String name, int id) {
        this.name = name;
        this.id = id;
    }
}
```

name
→ parameter

this.name
→ object field

Parameterized constructors allow each object to start with different values.

⑤ this Keyword

this refers to the current object.

```
class Student {
    String name;

    Student(String name) {
        this.name = name;
    }
}
```

this.name
(instance variable)

name
(constructor parameter)

⑥ Why do we use this ?

1. Resolve naming conflicts between fields and parameters.
2. Refer to the current object.
3. Call another constructor using **this(...)**.

⑦ Constructor Chaining

```
class Box {
    int size;

    Box() {
        this(25); // calls another constructor
    }

    Box(int size) {
        this.size = size;
    }
}
```

Constructor A

this(...)

Constructor B

this(...) must be the FIRST statement inside a constructor.

★ INTERVIEW SURPRISE

Q: Is this a constructor?

```
void Student() {
}
```

A: NO.

Because it has a return type (void), it is a method.

CONSTRUCTOR = NO RETURN TYPE

⑧ Constructor vs Method

CONSTRUCTOR	METHOD
Initializes object	Performs an operation
Same name as class	Can have any valid name
No return type	Has return type or void
Runs during object creation	Called explicitly

⑩ Small Example + Output

```
class User {
    String name;

    User(String name) {
        this.name = name;
    }

    void show() {
        System.out.println("User: " + name);
    }
}

public class Demo {
    public static void main(String[] args) {
        User u = new User("Arun");
        u.show();
    }
}
```

Output:

User: Arun



REMEMBER

- new → creates object
- Constructor → initializes object
- this → current object
- this(...) → calls another constructor

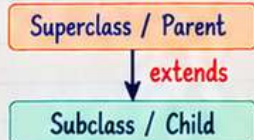


13. INHERITANCE

Reuse existing behavior by creating a relationship between classes.

① What is Inheritance?

Inheritance allows a subclass to reuse accessible members of a superclass.



- Promotes code reuse.
- Represents an **IS-A** relationship.
- Supports method overriding.
- Helps build class hierarchies.

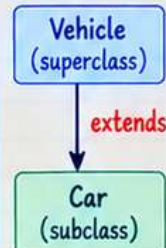
② Basic Syntax

```

class Vehicle {
    void move() {
        System.out.println("Moving");
    }
}

class Car extends Vehicle {
    void drive() {
        System.out.println("Driving");
    }
}
  
```

Car inherits the accessible **move()** method from Vehicle.



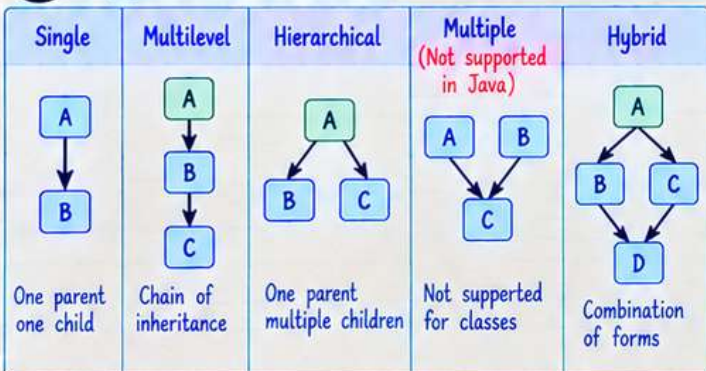
③ IS-A Relationship

- Dog **IS-A** Animal
- Car **IS-A** Vehicle
- Manager **IS-A** Employee

✗ Engine **IS NOT-A** Car
Engine is a component of Car
→ **HAS-A** relationship.

Inheritance models an **IS-A** relationship, not a **HAS-A** relationship.

④ Types of Inheritance



- Java does not support multiple inheritance of classes.
- Interfaces can provide multiple inheritance of type.

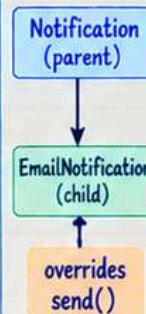
⑤ Method Overriding

When a subclass provides its own implementation of an inherited instance method with the same signature, the method is overridden.

```

class Notification {
    void send() {
        System.out.println("Sending notification");
    }
}

class EmailNotification extends Notification {
    @Override
    void send() {
        System.out.println("Sending email");
    }
}
  
```



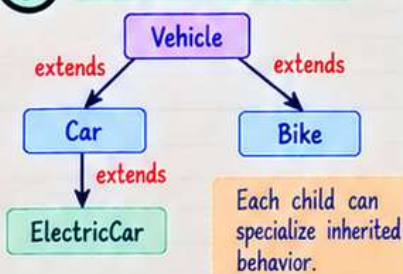
⑥ super Keyword

super refers to the immediate superclass context.

super();
→ calls parent constructor
super.send();
→ calls parent method
super.value;
→ accesses an accessible parent field

Used to access members of the parent class.

⑦ Inheritance Diagram



Each child can specialize inherited behavior.

⑧ Rules of Inheritance

- ✓ Constructors are not inherited.
- ✓ Private members are not directly accessible in subclasses.
- ✓ A class can extend only one class.
- ✓ A subclass can override eligible inherited instance methods.
- ✓ **super** can access superclass members where permitted.

⑨ extends vs implements

extends	implements
Used for class inheritance	Used for interface implementation
Extends a class (one superclass only)	Implements an interface (multiple interfaces allowed)
Example: class Car extends Vehicle	Example: class Car implements Drivable

⑩ Quick Example

```

class Person {
    void introduce() {
        System.out.println("I am a person");
    }
}

class Developer extends Person {
    void code() {
        System.out.println("Writing Java");
    }
}
  
```

```

public class Demo {
    public static void main(String[] args) {
        Developer d = new Developer();
        d.introduce();
        d.code();
    }
}
  
```

Output:

I am a person
Writing Java

🎯 INTERVIEW SURPRISE

Q: Does Java support multiple inheritance?

A: Java does **NOT** allow a class to extend multiple classes. A class can directly extend only one superclass. Interfaces can be implemented multiple times.

★ REMEMBER

- **extends** → class inheritance
- **super** → superclass context
- **IS-A** → inheritance relationship
- **HAS-A** → composition relationship
- **Override** → subclass-specific implementation

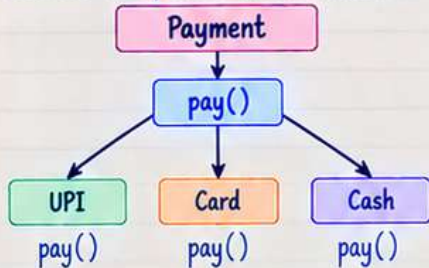


14. POLYMORPHISM

Same operation. Different behavior.

① What is Polymorphism ?

Polymorphism allows the same method name or interface to represent different behavior.



One common operation → different implementations.

② Types of Polymorphism

Compile-time Polymorphism (Method Overloading)

- Same method name
- Different parameter lists
- Resolved by compiler
- Occurs within the same class

OVERLOADING
→ COMPILE TIME

Runtime Polymorphism (Method Overriding)

- Same method signature
- Subclass provides implementation
- Resolved at runtime
- Occurs in inheritance

OVERRIDING
→ RUNTIME

③ Method Overloading

Multiple methods can have the same name when their parameter lists are different.

```

class Printer {
    void print(int value) {
        System.out.println(value);
    }
    void print(String value) {
        System.out.println(value);
    }
    void print(int a, int b) {
        System.out.println(a + b);
    }
}
  
```

```

print(int)
print(String)
print(int, int)
  
```

Compiler selects matching signature

④ What Counts as Different ?

Valid Overloading ✓

- add(int, int)
- add(double, double)
- add(int, int, int)
- add(String, String)

Invalid ✗

- add(int, int)
 - add(int, int)
- (Same parameter list is not allowed)

Return type alone cannot distinguish overloaded methods.

⑤ Method Overriding

A subclass provides its own implementation of an inherited method with the same signature.

```

class Report {
    void generate() {
        System.out.println("Generic report");
    }
}
class SalesReport extends Report {
    @Override
    void generate() {
        System.out.println("Sales report");
    }
}
  
```

```

Report r = new SalesReport();
r.generate();
  
```

Report (reference type)

SalesReport (actual object)

r.generate()

Sales report

⑥ How Runtime Polymorphism Works

Report r (superclass reference)

SalesReport object (actual object)

r.generate()

SalesReport.generate()

The overridden instance method is selected according to the actual object at runtime.

It is the **ACTUAL OBJECT** that decides which method to execute.

⑦ Upcasting

```

SalesReport s = new SalesReport();
Report r = s; // upcasting
  
```

Report (reference) → SalesReport (object)

Parent reference can refer to child object.

Assigning a subclass object to a superclass reference is called upcasting.

⑧ Downcasting

```

Report r = new SalesReport();
SalesReport s = (SalesReport) r;
  
```

Report reference

(SalesReport)

SalesReport object

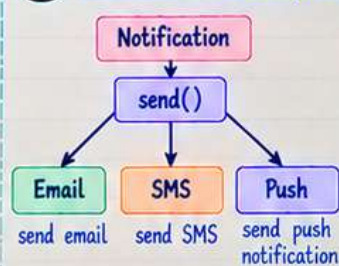
Downcasting converts a superclass reference to a subclass type when the actual object is compatible.

⚠ An invalid cast can cause **ClassCastException**.

⑨ Overloading vs Overriding

Feature	Overloading	Overriding
Method name	Same	Same
Parameters	Different	Same
Return type	Can be different	Must be same (or covariant)
Where it occurs	Same class	Parent - Child
When resolved	Compile time	Runtime
Key point	Compile time (Static)	Runtime (Dynamic)

⑩ Real-World Example



Same operation, specialized behavior.



INTERVIEW SURPRISE

Q: What happens here?

```

Notification n = new EmailNotification();
n.send();
  
```

A: The reference type is Notification, but the actual object is EmailNotification. If **send()** is overridden, the EmailNotification implementation executes.



REMEMBER

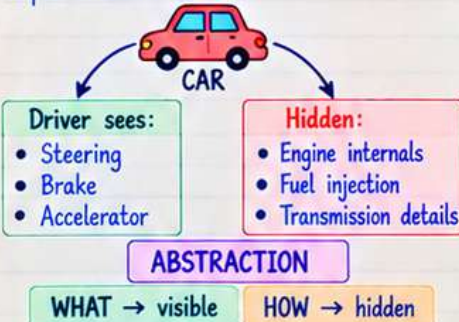
- Overloading → same name + different parameters + compile time
- Overriding → same signature + inheritance + runtime
- Upcasting → child object → parent reference
- Downcasting → parent reference → child type
- Polymorphism → same operation, different behavior

15. ABSTRACTION + INTERFACES

Expose what matters. Hide unnecessary implementation details.

① What is Abstraction ?

Abstraction focuses on what an object does while hiding unnecessary implementation details.



② Why use Abstraction ?

- ✓ Reduces unnecessary complexity
- ✓ Separates interface from implementation
- ✓ Makes systems easier to change
- ✓ Helps define common behavior

Focus on essential features, not the internal working.

③ Abstract Class

An abstract class is a class that cannot be instantiated directly and can contain abstract and concrete methods.

```
abstract class Shape {
    abstract double area();
    void display() {
        System.out.println("Shape");
    }
}
```

abstract class

→ cannot be instantiated

abstract method

→ no body (subclass must implement)

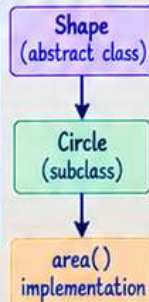
concrete method

→ has a body

④ Implementing an Abstract Class

```
class Circle extends Shape {
    double radius = 5;
    @Override
    double area() {
        return Math.PI * radius * radius;
    }
}
```

Subclass provides the implementation of the abstract method.



⑤ What is an Interface ?

An interface defines a contract that implementing classes agree to follow.

```
interface Printable {
    void print();
}

class Invoice implements Printable {
    public void print() {
        System.out.println("Invoice");
    }
}
```



⑥ Interface Features

- ✓ A class implements an interface.
- ✓ A class can implement multiple interfaces.
- ✓ Interfaces support abstraction through contracts.
- ✓ Interface methods can include abstract methods and (depending on Java version) default/static/private methods.
- ✓ Interface fields are implicitly public, static and final.

⑦ Abstract Class vs Interface

Feature	Abstract Class	Interface
Purpose	Provide common behavior	Define a contract
Fields	Can have instance fields	Fields are public, static, final
Constructors	Can have constructors	Cannot have constructors
Methods	Can have abstract + concrete methods	Methods are abstract by default (can also have default/static)
How to use	Extended using extends	Implemented using implements
Inheritance	A class can extend only one class	A class can implement multiple interfaces

⑧ Multiple Interfaces

```
interface Printable {
    void print();
}

interface Shareable {
    void share();
}

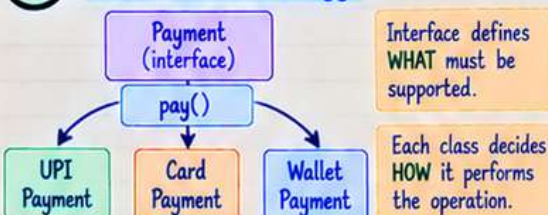
class Invoice implements Printable, Shareable {
    public void print() {
        System.out.println("Printing");
    }
    public void share() {
        System.out.println("Sharing");
    }
}
```

⑨ Default Method

```
interface Logger {
    default void log() {
        System.out.println("Logging...");
    }
}
```

A default interface method can provide an implementation that implementing classes may inherit.

⑩ Real-World Analogy



⑪ Important Difference

Term	Meaning
Abstraction	Concept / design principle
Abstract class	One Java mechanism for achieving abstraction
Interface	Another Java mechanism for defining contracts and supporting abstraction

🎯 INTERVIEW SURPRISE

Q: Can a class implement multiple interfaces?

A: YES.

Example:

```
class Payment implements
    OnlinePay, Refundable { }
```

A Java class can extend only one class but can implement multiple interfaces.

📌 REMEMBER

- Abstraction → hide unnecessary details
- Abstract class → partial abstraction + shared implementation
- Interface → contract

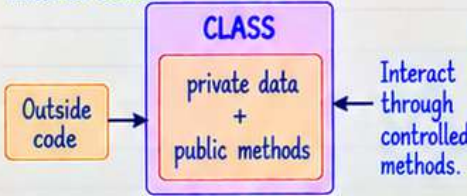
- extends → class inheritance
- implements → interface implementation
- default method → method with implementation (in interface)

16. ENCAPSULATION + ACCESS MODIFIERS

Protect data and control how other code can access it.

① What is Encapsulation ?

Encapsulation means bundling data and the methods that operate on that data inside a class while controlling access to the internal state.



DATA + METHODS + CONTROLLED ACCESS

② Why Encapsulation ?

- ✓ Protect internal state
- ✓ Prevent unwanted direct modification
- ✓ Validate incoming values
- ✓ Make implementation easier to change
- ✓ Improve maintainability

Keeps the object in a safe and consistent state.

③ Basic Example

```
class BankAccount {
    private double balance;

    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
        }
    }

    public double getBalance() {
        return balance;
    }
}
```

private balance
→ cannot be directly accessed from unrelated code

deposit()
→ controlled modification

getBalance()
→ controlled reading

④ Access Modifiers

Modifier	Same Class	Same Package	Subclass	Other Package
private	Yes	No	No direct access	No
default (package-private)	Yes	Yes	No outside package	No
protected	Yes	Yes	Yes, subject to protected access rules	Limited access through subclass context
public	Yes	Yes	Yes	Yes, when the containing type is accessible

⑤ private

```
private int pin;
```

- Accessible only within the declaring class.

MOST RESTRICTIVE

⑥ default / package-private

```
int score;
```

- When no access modifier is written, the member has package-private access.
- Accessible within the same package.

NO KEYWORD

⑦ protected

```
protected int points;
```

- Accessible within the same package and also available to subclasses, subject to Java's protected-access rules.

⑧ public

```
public void display() { }
```

- Accessible from any location where the containing class/member is accessible.

WIDEST MEMBER ACCESS

⑨ Encapsulation vs Data Hiding

ENCAPSULATION	DATA HIDING
• Bundles data and behavior • Controls how state is accessed	• Restricts direct visibility/access to implementation details

private fields + public methods are a common way to implement encapsulated state.

⑩ Getters & Setters

```
class User {
    private String name;

    public String getName() {
        return name;
    }

    public void setName(String name) {
        if (name != null && !name.isBlank()) {
            this.name = name;
        }
    }
}
```

private field

getter()

→ read

setter()

→ controlled update

Setter can validate data before changing the field.

⑪ Access Modifier Quick View



Think of this as increasing accessibility, but protected has special subclass rules.

🔥 INTERVIEW SURPRISE

- Q: Which access modifier is the most restrictive?
A: **private**
- Q: What is the default access level when no modifier is specified?
A: **package-private**

NO KEYWORD → PACKAGE-PRIVATE

★ REMEMBER

- **private** → same class
- **package-private** → same package
- **protected** → same package + permitted subclass access
- **public** → widest access
- **Encapsulation** → protect/control object state
- **Getter** → read
- **Setter** → controlled update

17. EXCEPTION HANDLING

Handle unexpected situations without crashing the normal flow of a program.

① What is an Exception?

An exception is an abnormal event that occurs during program execution and interrupts the normal flow of instructions.

Examples :

- Dividing by zero
- Accessing an invalid array index
- Using a null reference
- Opening a missing file

EXCEPTION = RUNTIME PROBLEM

② Why Exception Handling?

- ✓ Prevent sudden program termination
- ✓ Separate error-handling logic from normal logic
- ✓ Display meaningful error information
- ✓ Allow the program to recover when possible
- ✓ Improve application reliability

Handle errors gracefully and keep the application stable.

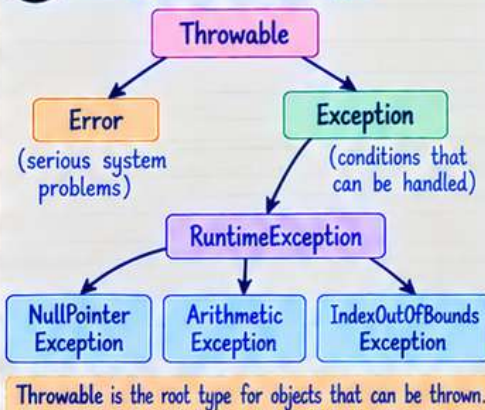
③ Basic try-catch

```
try {
    int result = 20 / 0;
    System.out.println(result);
}
catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero");
}
```

Flow :



④ Exception Hierarchy



⑤ Checked vs Unchecked Exceptions

CHECKED EXCEPTIONS	UNCHECKED EXCEPTIONS
• Checked by compiler • Usually must be caught or declared • Often related to external resources	• Occur during runtime • Compiler does not require handling • Often caused by programming mistakes
Examples: IOException SQLException	Examples: NullPointerException ArithmeticException ArrayIndexOutOfBoundsException

⑥ throw vs throws

throw	throws
Actually throws an exception object.	Declares that a method may pass an exception to its caller.
Example: <pre>throw new IllegalArgumentExcep tion("Age cannot be negative");</pre>	Example: <pre>void readFile() throws IOException { // code }</pre>
throw → inside method	throws → method declaration

⑦ finally

finally is used for cleanup code that should normally execute whether an exception occurs or not.

```
try {
    System.out.println("Processing");
} catch (Exception e) {
    System.out.println("Handled");
} finally {
    System.out.println("Cleanup");
}
```

finally usually runs after try/catch, even when an exception is handled.

⑧ Multiple Catch

```
try {
    int[] nums = {10, 20};
    System.out.println(nums[5]);
} catch (ArithmeticException e) {
    System.out.println("Math problem");
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Invalid index");
}
```

Place more specific catches before broader catches.

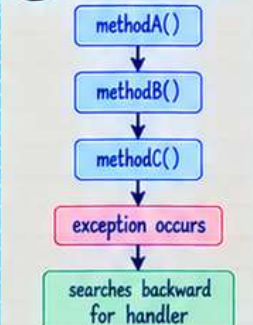
⑨ Custom Exception

```
class InvalidAgeException
extends Exception {
    InvalidAgeException(String
message) {
        super(message);
    }
}

if (age < 18) {
    throw new InvalidAgeException
("Not eligible");
}
```

Custom exceptions allow an application to represent domain-specific problems.

⑩ Exception Propagation



If a method does not handle an exception, it can propagate to its caller.

⑪ Best Practices

- ✓ Catch specific exceptions.
- ✓ Do not silently ignore exceptions.
- ✓ Use meaningful error messages.
- ✓ Use **finally** or try-with-resources for cleanup when appropriate.
- ✓ Do not use exceptions as normal control flow.



INTERVIEW SURPRISE

Q: What is the difference between **throw** and **throws**?

A: **throw** → throws one exception object explicitly.
throws → declares possible exceptions in the method signature.

throw = ACTION | throws = DECLARATION



REMEMBER

- **try** → risky code
- **catch** → handle exception
- **finally** → cleanup
- **throw** → explicitly throw exception
- **throws** → declare possible exception
- **Checked** → compiler involvement
- **Unchecked** → runtime exceptions
- **Exception** → abnormal event

18. COLLECTION FRAMEWORK

Choose the right data structure for the job.

① What is Collection Framework?

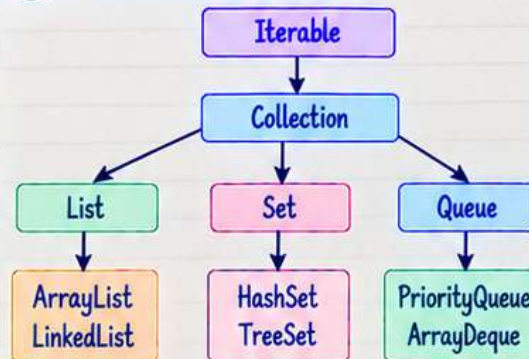
The Java Collections Framework provides interfaces, implementations and algorithms for storing and manipulating groups of objects.

DATA STRUCTURE + READY-MADE OPERATIONS

Commonly used collections :

- List - ordered collection
- Set - unique elements
- Queue - FIFO / queue behavior
- Map - key-value pairs

② Collection Hierarchy



Map is **NOT** a subtype of Collection.



③ List

Ordered collection that allows duplicate elements.

Implementation	Key idea
ArrayList	• Fast random access • Good general-purpose choice
LinkedList	• Linked structure • Efficient insertion/removal at ends

```

List<String> names = new ArrayList<>();
names.add("Asha");
names.add("Ravi");
names.add("Asha");
System.out.println(names);

```

Output: [Asha, Ravi, Asha]

ORDERED + DUPLICATES ALLOWED

④ Set

Collection that does not allow duplicate elements.

HashSet → no guaranteed iteration order

LinkedHashSet → preserves insertion order

TreeSet → keeps elements sorted

HashSet → uniqueness

LinkedHashSet → uniqueness + insertion order

TreeSet → uniqueness + sorted order

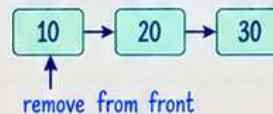
⑤ Queue

Used when elements are processed according to a particular queue policy (usually FIFO).

```

Queue<Integer> q = new ArrayDeque<>();
q.offer(10);
q.offer(20);
q.offer(30);
System.out.println(q.poll()); // 10

```



- **offer()** → add element
- **poll()** → remove element
- **peek()** → inspect front element

⑥ Map

Stores data as key-value pairs.

```

Map<Integer, String> users = new HashMap<>();
users.put(101, "Riya");
users.put(102, "Kiran");

```

Key	Value
101	→ Riya
102	→ Kiran

- Keys are unique.
- Values may repeat.

⑦ Common Map Implementations

Implementation	Key idea
HashMap	• General-purpose • No ordering guarantee
LinkedHashMap	• Maintains insertion order
TreeMap	• Sorts by keys

HashMap → fast general choice.
TreeMap → sorted keys.

⑧ Important Methods

List / Set	Map	Queue
add()	put()	offer()
remove()	get()	poll()
contains()	remove()	peek()
size()	containsKey()	-
clear()	containsValue()	-

⑨ Iterating Collections

```

// 1. for-each loop
for (String name : names) {
    System.out.println(name);
}

// 2. Iterator
Iterator<String> it = names.iterator();
while (it.hasNext()) {
    System.out.println(it.next());
}

// 3. forEach (Java 8+)
names.forEach(System.out::println);

```

Choose the approach that keeps the code clear.

⑩ When to Use What?

Need duplicates?	→ List
Need unique values?	→ Set
Need key → value?	→ Map
Need queue behavior?	→ Queue

Start from the required behavior, then choose the implementation.

⑪ Time Complexity Snapshot

Operation	ArrayList	HashSet	HashMap
get(index)	Fast O(1)	—	—
add / search	Depends	Average O(1)	Average O(1)
sorted operations	—	TreeSet	TreeMap



INTERVIEW SURPRISE

Q: Is Map a child of Collection?

A: NO.

Map stores key-value mappings and is separate from the Collection hierarchy.

Collection → elements | Map → key + value



QUICK CHOICE

- ArrayList → indexed ordered data
- HashSet → unique data
- TreeSet → sorted unique data
- HashMap → key-value lookup
- TreeMap → sorted keys
- ArrayDeque → queue/deque operations



19. MULTITHREADING + FILE HANDLING

Run tasks concurrently and work with persistent data.

PART - A : MULTITHREADING

① What is a Thread ?

A thread is a lightweight path of execution inside a process.

PROCESS

Thread 1 → Task A
Thread 2 → Task B
Thread 3 → Task C

Multiple threads can make progress concurrently.

Same process, multiple threads working on different tasks.

② Process vs Thread

Feature	Process	Thread
Execution	Independent execution unit	Runs inside a process
Resources	Own memory and resources	Shares process resources
Cost	More expensive	Lightweight
Communication	Inter-process communication	Directly shares memory

THREADS SHARE MEMORY
(within the same process)

③ Creating a Thread

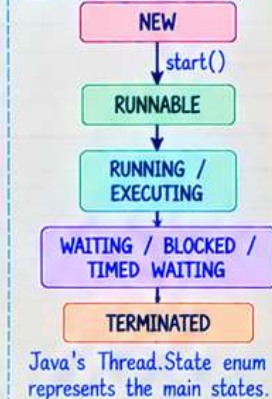
```
class Task implements Runnable {
    public void run() {
        System.out.println("Task running");
    }
}
```

```
Thread t = new Thread(new Task());
t.start(); // starts a new thread
```

Call `start()` to begin a new thread. Calling `run()` directly is just a normal method call.



④ Thread Lifecycle



⑤ Common Thread Methods

Method	Purpose
<code>start()</code>	Begins thread execution
<code>run()</code>	Contains the task logic
<code>sleep(ms)</code>	Pauses current thread temporarily
<code>join()</code>	Waits for another thread to finish
<code>interrupt()</code>	Requests interruption of a thread

start() ≠ run()

`start()` creates a new thread, `run()` just calls the method.

⑥ Synchronization

When multiple threads access shared mutable state, synchronization can help prevent race conditions.

```
class Counter {
    private int count;
    synchronized void increment() {
        count++;
    }
}
```



SHARED STATE → POSSIBLE RACE CONDITION

⑦ Executor Service

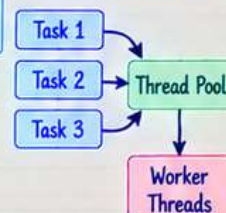
```
ExecutorService executor =
    Executors.newFixedThreadPool(2);
executor.submit() → {
    System.out.println("Task executed");
};
executor.shutdown();
```

- ExecutorService manages a pool of worker threads instead of creating a new thread for every task.
- Helps in better resource management and scalability.

TASK → THREAD POOL

Why use ?

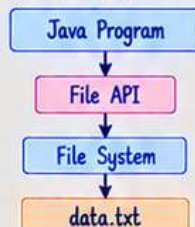
- Reuses threads
- Improves performance
- Simplifies thread management



PART - B : FILE HANDLING

⑧ What is File Handling ?

File handling allows Java programs to create, read, write and manage data stored in files.



Data remains stored even after the program ends.

⑨ Modern File API (java.nio.file)

```
Path path = Path.of("notes.txt");
Files.writeString(path, "Java is powerful");
String text = Files.readString(path);
System.out.println(text);
```

- `java.nio.file` provides convenient modern APIs for file operations.
- Path** → represents a file path.
- Files** → provides file operations.

Modern API is easier and more flexible than older file classes.

⑩ Reading + Writing

```
WRITE (Program → File)
Files.writeString(
    Path.of("data.txt"),
    "Hello Java"
);

READ (File → Program)
String data =
    Files.readString(
        Path.of("data.txt")
    );
```

Writing stores data in the file. Reading fetches data from the file.

⑪ Buffered I/O



- Buffered I/O can reduce the overhead of frequent small read/write operations.
- Common classes:
 - `BufferedReader`
 - `BufferedWriter`

⑫ Try-With-Resources

```
try (BufferedReader br =
    Files.newBufferedReader(
        Path.of("data.txt"))) {
    System.out.println(br.readLine());
} catch (IOException e) {
    e.printStackTrace();
}
```

- Resources declared in try-with-resources are automatically closed.

AUTO CLOSE
(no need to close manually)

🔥 INTERVIEW SURPRISE

Q: What is the difference between `start()` and `run()` ?

A: `start()` → starts a new thread of execution.

`run()` → executes the method body in the current thread when called directly.

start() = NEW THREAD | run() = CURRENT THREAD

📌 REMEMBER

- Thread → path of execution
- `start()` → starts thread
- `sleep()` → temporary pause
- `join()` → wait for thread
- `synchronized` → protect critical shared access

- ExecutorService → manage thread pools
 - Path → represents file path
 - Files → file operations
 - try-with-resources → automatic resource closing
- Collection of these concepts helps build real-world applications.

20. JAVA INTERVIEW QUESTIONS + CODING CHEAT SHEET

Last-minute Java revision – concepts + code you should remember.

Practice
Revise
Get Hired
...

PART A : TOP INTERVIEW QUESTIONS

1 Core Java Questions

- Q1.** Why is Java platform independent?
→ Compiled to bytecode, which runs on JVM for the target platform.
- Q2.** JDK vs JRE vs JVM?
→ JDK = tools + runtime
→ JRE = runtime environment
→ JVM = executes bytecode
- Q3.** Why is Java not purely OOP?
→ Supports primitive data types (int, char, boolean, etc).
- Q4.** Difference between == and equals()?
→ == compares primitive values or reference identity.
→ equals() compares logical equality (when overridden).

2 OOP Questions

- Q5.** What is inheritance?
→ Reusing superclass behavior in subclass.
- Q6.** Overloading vs overriding?
→ Overloading: same method name, different parameters.
→ Overriding: subclass provides its own implementation.
- Q7.** What is polymorphism?
→ One reference can represent different implementations.
- Q8.** What is encapsulation?
→ Bundling state and behavior while controlling access.
- Q9.** Abstract class vs interface?
→ Abstract class: share state and implementation.
→ Interface: defines a contract (multiple implementation).

3 String + Memory Questions

- Q10.** String vs StringBuilder?
→ String is immutable.
→ StringBuilder is mutable.
- Q11.** What is String immutability?
→ Once created, its character sequence cannot be changed.
- Q12.** Stack vs Heap?
→ Stack: execution frames, local variables.
→ Heap: objects and arrays managed by JVM.
- Q13.** What is garbage collection?
→ JVM automatically reclaims memory of unreachable objects.

4 Exception + Thread Questions

- Q14.** Checked vs unchecked exception?
→ Checked: compile-time checking.
→ Unchecked: RuntimeException subclasses (no need to catch).
- Q15.** throw vs throws?
→ throw: explicitly throws an exception object.
→ throws: declares possible exceptions in method signature.
- Q16.** start() vs run()?
→ start(): begins a new thread.
→ run(): direct method call (just normal execution).
- Q17.** Why synchronization?
→ To coordinate access to shared mutable state & avoid race conditions.

5 Collection Questions

- Q18.** ArrayList vs LinkedList?
→ ArrayList: fast indexed access.
→ LinkedList: efficient insertion/removal at ends.
- Q19.** HashSet vs TreeSet?
→ HashSet: unique elements, no order guarantee.
→ TreeSet: unique elements, sorted order.
- Q20.** HashMap vs TreeMap?
→ HashMap: general-purpose.
→ TreeMap: sorted by keys.
- Q21.** Can HashMap have null key?
→ Yes, one null key and multiple null values are allowed.

PART B : COMMON CODING SNIPPETS

6 Reverse a String

STRING REVERSAL

```
String s = "java";
String rev = "";

for (int i = s.length() - 1; i >= 0; i--) {
    rev += s.charAt(i);
}

System.out.println(rev);
```

Output: avaj

7 Palindrome Check

TWO POINTERS

```
String s = "level";
boolean palindrome = true;

for (int i = 0, j = s.length() - 1; i < j; i++, j--) {
    if (s.charAt(i) != s.charAt(j)) {
        palindrome = false;
        break;
    }
}

System.out.println(palindrome);
```

Output: true

8 Frequency Using HashMap

COUNT FREQUENCY

```
Map<Character, Integer> freq =
    new HashMap<>();

for (char c : "banana".toCharArray()) {
    freq.put(c, freq.getOrDefault(c, 0) + 1);
}
```

Result:

b → 1
a → 3
n → 2

9 Swap Two Numbers

BASIC LOGIC

```
int a = 10;
int b = 20;

int temp = a;
a = b;
b = temp;

System.out.println(a + " " + b);
```

Output: 20 10

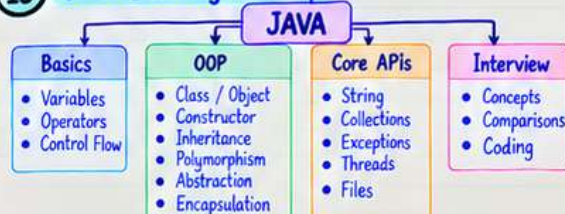
PART C : MUST-KNOW SYNTAX

10 Java Quick Syntax Box

Array	Loop (for-each)	Method
<code>int[] arr = {1, 2, 3};</code>	<code>for (int x : arr) { System.out.println(x); }</code>	<code>static int add(int a, int b) { return a + b; }</code>
Object	Exception	Thread
<code>Student s = new Student();</code>	<code>try { // risky code } catch (Exception e) { // handle }</code>	<code>new Thread() → task().start();</code>

PART F : FINAL REVISION MAP

13 Java Learning Roadmap



PART-D : RAPID COMPARISON

11 Key Concepts at a Glance

Concept	Remember
String	immutable
StringBuilder	mutable
ArrayList	ordered + duplicates
HashSet	unique elements
HashMap	key-value
Overloading	compile time
Overriding	runtime
abstract class	shared abstraction
interface	contract
private	class-level access

Before Your Java Interview

Remember these first:

- OOP fundamentals
- String + immutability equals() + hashCode()
- Collections
- Exception handling
- Multithreading basics
- JVM / JDK / JRE
- Access modifiers
- Overloading vs overriding
- Practice coding without IDE assistance

PART-E : INTERVIEW TRAPS

12 Tricky Questions

- Q:** Can a constructor be inherited?
A: NO.
- Q:** Can a static method be overridden?
A: It is **hidden**, not overridden.
- Q:** Can an abstract class have a constructor?
A: YES.
- Q:** Can an interface have implementation methods?
A: YES – default, static and private methods.
- Q:** Can a class extend multiple classes?
A: NO.
- Q:** Can a class implement multiple interfaces?
A: YES.

14 Final Message

Save this page for your Java interview revision.

FOLLOW @codewith_govind

for Java • DSA • System Design • Interview Prep

"Consistent Learning Creates Real Opportunities"